



Gamified micro-learning app for technical micro-skills

Artefact Design and Test Plan

Student Id: 2417733

Student Name: Rojin Singh Mahat

Supervisor: M.D. Saddam Khan

Reader: Shiv Kumar Yadav

Section: L6CG10

Submitted on: 26-12-2025

Table of Contents

1. Introduction.....	1
1.1 Aims and Objectives	1
1.2 Problem Stated	1
1.3 System as a Solution	2
2. System Artifact	2
2.1 User Domain Features.....	2
2.2 Admin Domain Features	4
3. System Design	5
3.1 Overview	5
3.2 Functional Decomposition Diagram	5
3.3 Overall Use Case Diagram.....	6
3.4 Entity Relationship Diagram (ERD).....	8
3.5 Class Diagram	11
3.6 Overall System Activity Diagram.....	13
3.7 Overall System Sequence Diagrams	14
3.7.1 Sequence Diagram A – Payment-based Unlocking	14
3.7.2 Sequence Diagram B – Lesson Completion, Evaluation, and Rewards	16
4. Test Plan.....	17
4.1 Test Plan Overview	17
4.2 Testing Scope.....	17
4.3 Testing Approach.....	18
4.4 Test Environment.....	18
4.5 Test Cases (Core Functional Tests)	19
4.6 Test Execution Status.....	22
5. Work Breakdown Structure	30

1. Introduction

1.1 Aims and Objectives

Aim

The aim of this project is to design and develop a gamified learning platform that enhances learner engagement through structured lessons, quizzes, progress tracking, and reward mechanisms.

Objectives

- To design a role-based system supporting learners and administrators
- To implement structured learning content organized into quests and lessons
- To integrate quizzes for knowledge assessment and reinforcement
- To track learner progress and award rewards based on performance
- To provide administrative tools for managing learning content and users
- To ensure the system is scalable, maintainable, and aligned with software engineering best practices

1.2 Problem Stated

Traditional online learning platforms often struggle with low user engagement, limited motivation, and poor progress retention. Learners may lack clear incentives to complete lessons, while educators and administrators face difficulties in managing structured content and tracking learner performance effectively.

Many existing systems focus solely on content delivery without incorporating interactive elements such as quizzes, rewards, or tiered access control. This results in passive learning experiences that fail to sustain long-term learner interest and reduce overall learning effectiveness.

1.3 System as a Solution

The proposed system addresses these challenges by introducing a gamified learning environment that combines structured lessons, quizzes, progress tracking, and reward mechanisms. Learners interact with content through quests composed of lessons and assessments, promoting active participation and continuous engagement.

The system supports multiple user roles, enabling administrators to create, manage, and monitor learning content while allowing learners to track their progress through dashboards and rewards. By integrating tiered access control (payment-based unlocking of advanced content), feedback loops, and performance-based incentives, the platform provides both free-to-play engagement and sustainable monetization while delivering a more engaging and effective learning experience.

2. System Artifact

2.1 User Domain Features

The User (Learner) domain focuses on content consumption, assessment, and progress tracking.

User Authentication

Learners can register and log into the system using secure authentication. This ensures personalized access to learning progress and rewards.

Browse Learning Content

Learners can browse available quests and lessons, categorized by difficulty level (Beginner, Intermediate, Advanced), enabling structured progression.

Difficulty Selection

Learners select a difficulty level before starting a quest. Certain difficulty levels may be locked until prerequisites are met or conditions are satisfied.

Lesson Participation

Each quest consists of lessons containing instructional content. Learners progress sequentially through lessons to maintain learning flow.

Quiz Attempt

After completing a lesson, learners attempt quizzes to assess understanding. Each quiz contains multiple-choice questions with a single correct answer.

Feedback and Retry

Immediate feedback is provided after quiz attempts. Incorrect answers allow learners to retry lessons, reinforcing learning through repetition.

Progress Tracking

Learner progress, completed lessons, earned XP, and rewards are tracked and displayed on a personalized dashboard.

Rewards and XP System

Learners earn XP and rewards upon successful completion of quizzes and quests, motivating continued engagement.

2.2 Admin Domain Features

The Admin domain enables system control, content management, and monitoring.

Admin Authentication

Administrators authenticate with elevated privileges to access management features.

Quest Management

Admins can create, edit, update, and delete quests. Each quest contains metadata such as title, description, difficulty, total XP, and reward badges.

Lesson Management

Admins manage lessons within quests, including lesson content, order, XP rewards, and associated quizzes.

Quiz Management

Admins define quiz questions, answer options, and correct answers to ensure accurate assessment.

User Monitoring

Admins can view learner progress, quest completion status, and engagement metrics for evaluation purposes.

Content Control

Admins ensure content consistency, correctness, and progression logic across difficulty levels.

3. System Design

3.1 Overview

Overall system-level diagrams present the system's structure and behavior from complementary perspectives, enabling stakeholders to understand the solution without digging into code.

Included artefacts:

- Functional Decomposition Diagram
- Overall Use Case Diagram
- Entity Relationship Diagram (ERD)
- Class Diagram
- Key Sequence Diagrams (payment unlocking, lesson completion)

Each diagram focuses on a distinct concern like features, user interactions, data model, code-level structure, and runtime flows providing a comprehensive architectural picture.

3.2 Functional Decomposition Diagram

The Functional Decomposition Diagram illustrates how the system is divided into major functional modules. The purpose of this diagram is to break down the system into manageable components, each responsible for a specific set of tasks.

At the highest level, the system is divided into the following core modules:

- User Authentication and Authorization
- Core Feature Management (e.g., quests, lessons, quizzes, or equivalent system features)

- Data Management and Persistence
- User Interface and Interaction
- Administration and System Control

Each module is further decomposed into sub-functions. For example, the authentication module includes user registration, login, token validation, and role-based access control. The feature management module handles creation, modification, deletion, and retrieval of system entities.

This decomposition ensures a modular architecture, making the system easier to maintain, test, and extend.

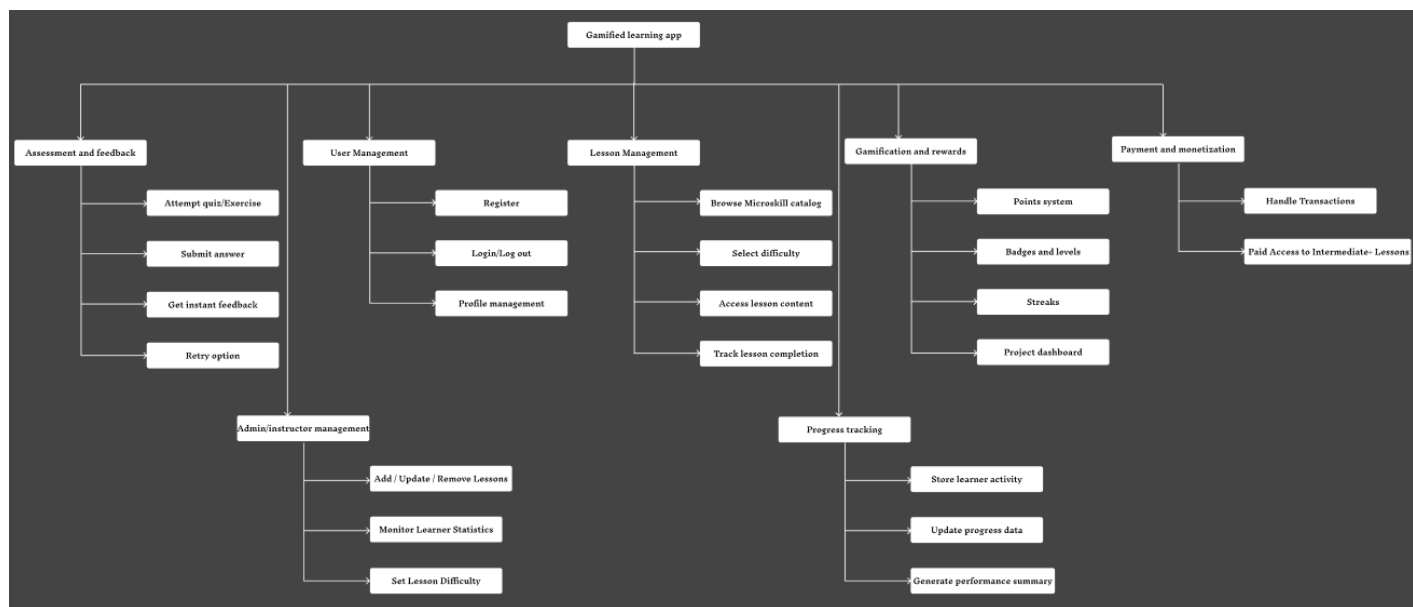


Figure 3.2.1 – Functional Decomposition Diagram

3.3 Overall Use Case Diagram

The Overall Use Case Diagram provides a high-level view of how different actors interact with the system. It identifies the system's primary users and the actions they can perform.

Typical actors include:

- User/Learner
- Administrator/Instructor

Each actor is associated with a set of use cases. For example, users may interact with core system features, while administrators have additional privileges such as managing content, users, or system configurations.

This diagram helps clarify system boundaries and ensures that all required user interactions are accounted for during development.

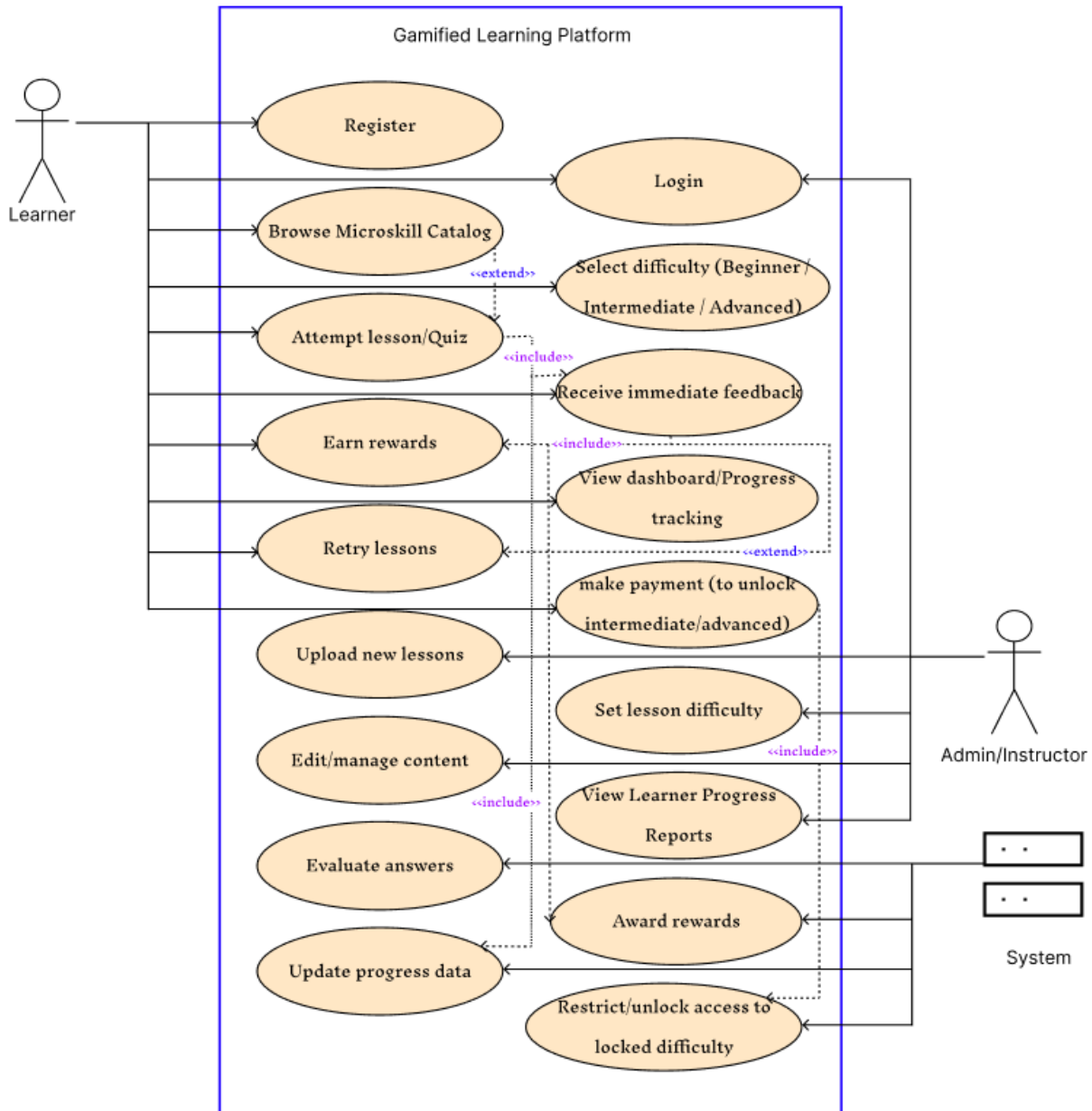


Figure 3.3.1 – Overall Use Case Diagram

3.4 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram represents the logical structure of the system's database. It defines the main entities, their attributes, and the relationships between them.

Key entities typically include:

- User
- Core Domain Entities (e.g., Quest, Lesson, Quiz, etc.)
- Supporting Entities (e.g., Progress, Rewards, etc.)

Relationships such as one-to-many and many-to-many are clearly defined to maintain data integrity and consistency. The ERD ensures that the database design aligns with the system's functional requirements and supports efficient data retrieval and storage.

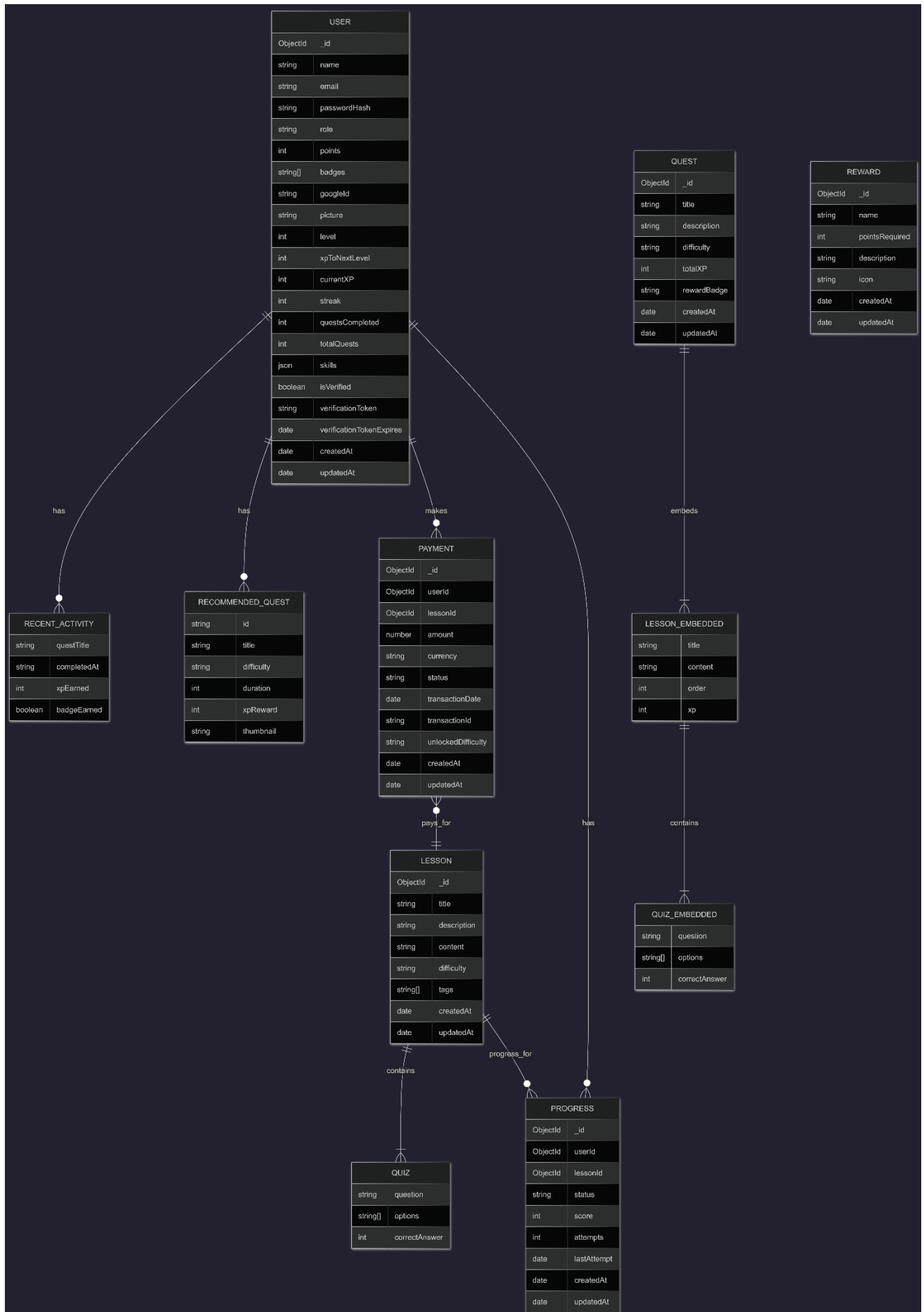


Figure 3.4.1 – Entity Relationship Diagram

3.5 Class Diagram

The class diagram mirrors the code-level structure:

- Classes map to Mongoose models: User, Quest, Lesson (standalone), Progress, Payment, Reward, plus embedded substructures for lessons/quizzes within quests.
- Composition:
 - Quest embeds EmbeddedLesson, which embeds EmbeddedQuiz
 - Lesson contains Quiz array
- Associations:
 - Progress associates User and Lesson
 - Payment associates User and Lesson and records unlockedDifficulty
- Attributes reflect current schemas (timestamps, enums, arrays, maps)

This artefact bridges high-level design and implementation, aiding developers in navigating models and their relationships.

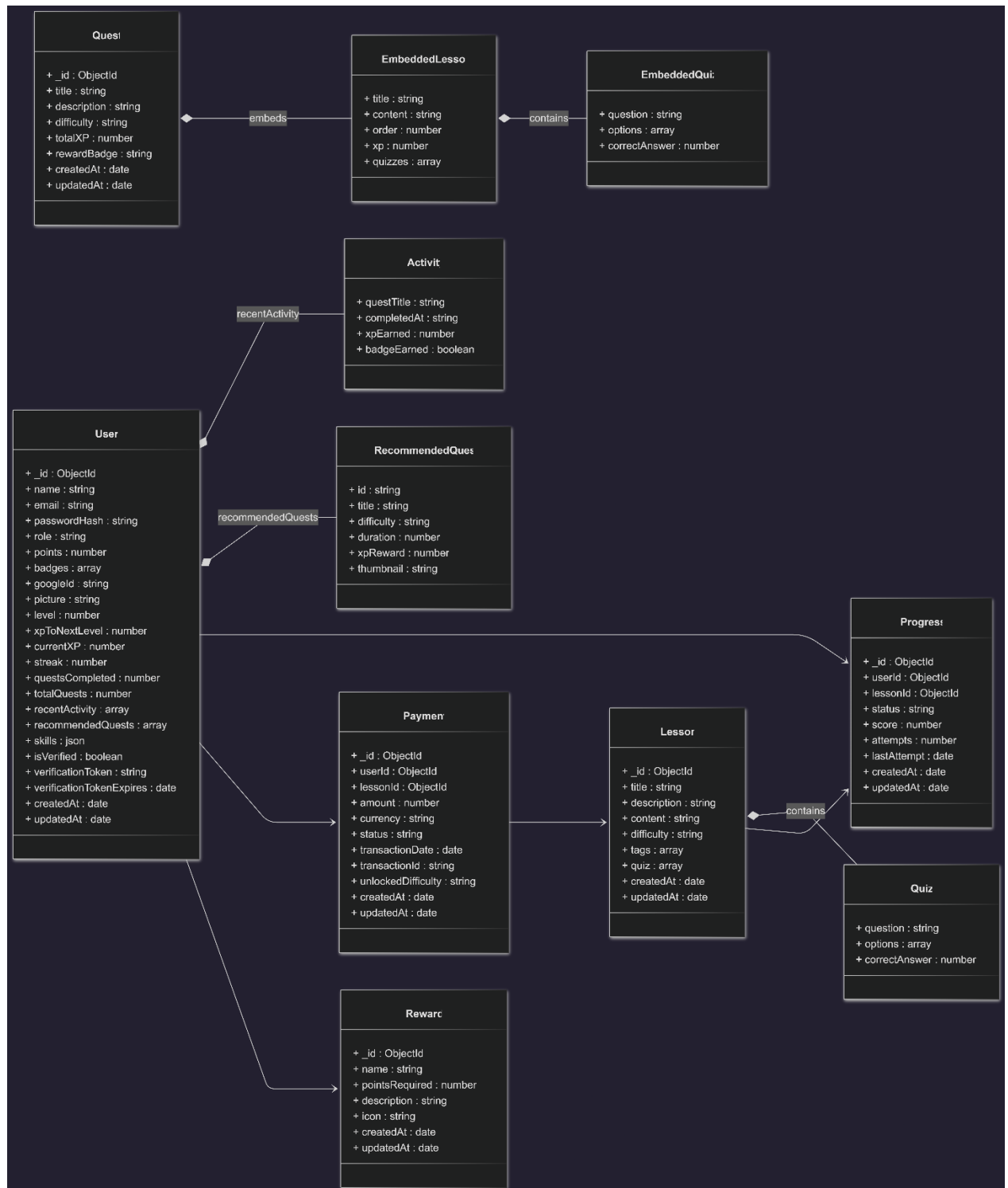


Figure 3.5.1 – Class Diagram

3.6 Overall System Activity Diagram

The Overall System Activity Diagram illustrates the end-to-end workflow of the learning platform, highlighting the interaction between the Learner and the System through clearly separated swimlanes. The diagram captures the main control flow, decision points, and system responses during a typical learning session.

The process begins confirming the successful authentication of the learner, after which the learner is able to browse the available course catalog. Once a suitable course is identified, the learner selects a difficulty level. The system supports three difficulty tiers: Beginner, Intermediate, and Advanced.

For difficulty levels that are restricted, the system performs a payment verification step. If payment is not completed, access to the selected difficulty level is denied. Upon successful payment, the system grants access to the locked content, allowing the learner to proceed.

After gaining access to the selected lesson, the learner attempts the associated quiz. The system evaluates the submitted answers and determines whether the response is correct. If the answer is incorrect, feedback is provided and the learner is given the option to retry the lesson. If the learner chooses not to retry, the workflow returns to the dashboard.

If the answer is correct, the system awards rewards and updates the learner's progress accordingly. Once progress is updated, the learner is redirected to the dashboard, concluding the learning cycle.

This activity diagram demonstrates the system's support for adaptive learning, progress tracking, reward mechanisms, and conditional access control. It provides a comprehensive view of both user-driven actions and automated system decisions, ensuring clarity in how learning content is

accessed and reinforced.

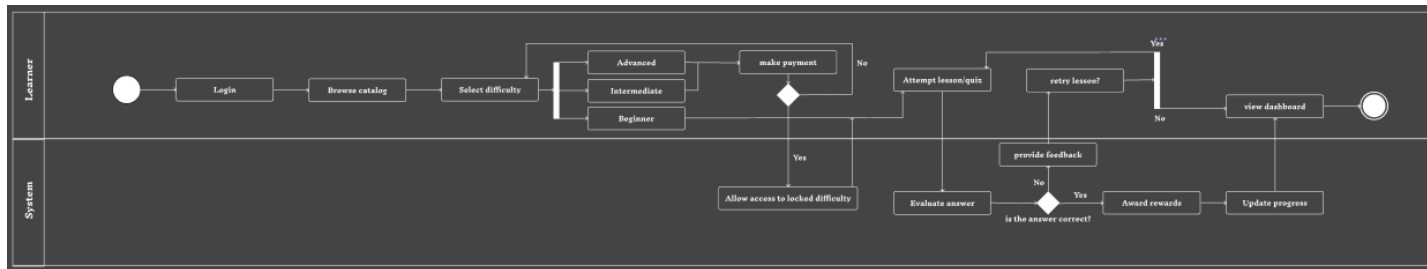


Figure 3.6.1 – Overall System Activity Diagram

3.7 Overall System Sequence Diagrams

This section provides system-level overviews aligned with the two key sequence diagrams:

- (1) Payment-based unlocking of intermediate/advanced content and
- (2) Lesson completion with evaluation and rewards.

Each overview summarizes actors, major steps, decision points, system responsibilities, and data/state updates to support traceability from requirements to implementation.

3.7.1 Sequence Diagram A – Payment-based Unlocking

Actors:

Learner, Frontend (React App), Backend API (Express), Payment Provider, Database.

Flow Summary:

Authenticate user → request to unlock lesson/difficulty → backend creates/validates payment intent → redirect/confirm with payment provider → on success, backend records Payment and grants access → UI updates progress and availability.

Key Decisions:

Is payment required for selected difficulty? Did payment succeed (provider callback/webhook)?

Are user and lesson valid and authorized?

Data Updates:

Create Payment record (status, providerRef, amount, unlockedDifficulty); update User access rights; log progress events.

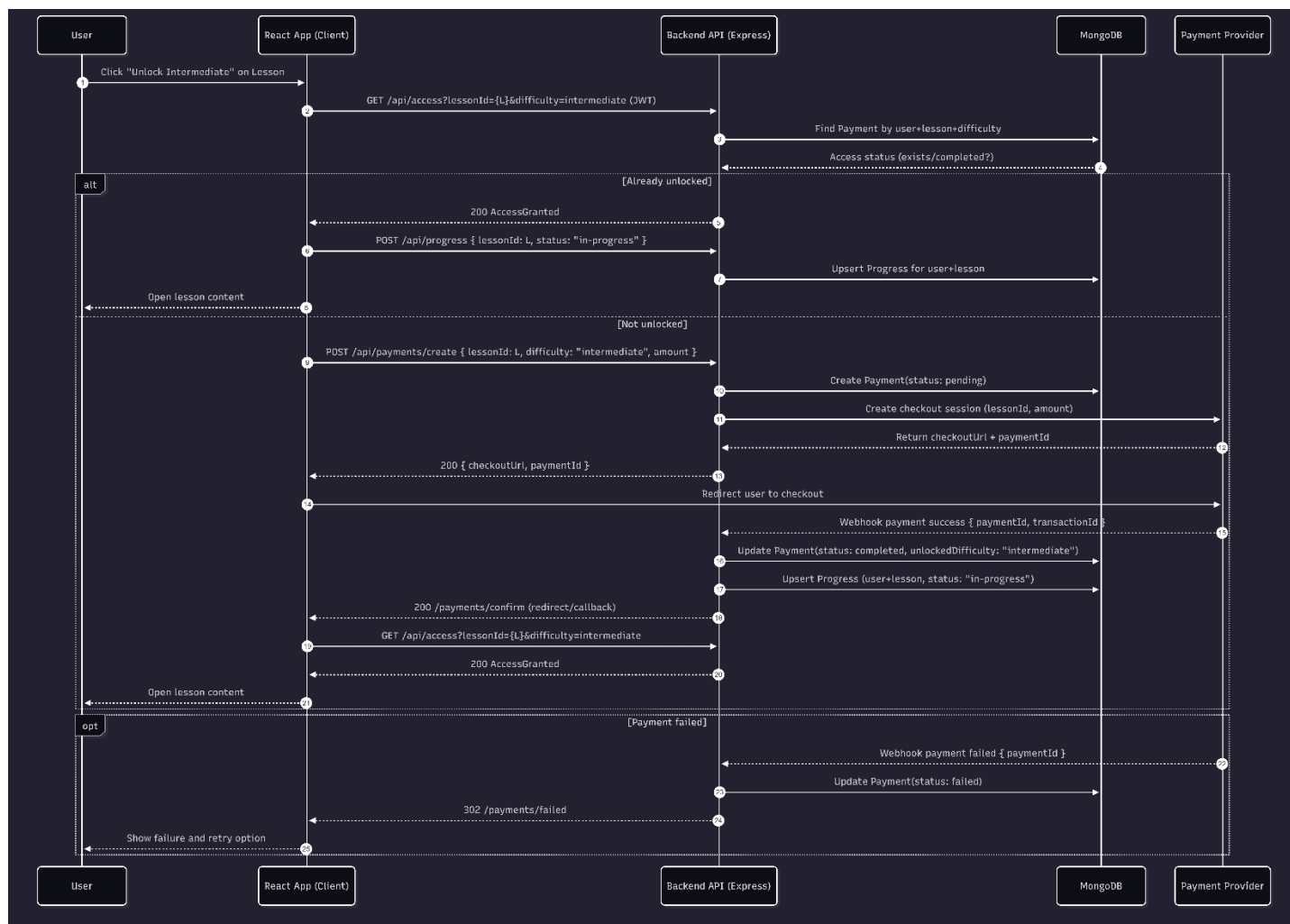


Figure 3.7.1.1: Sequence Diagram for Payment flow

3.7.2 Sequence Diagram B – Lesson Completion, Evaluation, and Rewards

Actors:

Learner, Platform UI, Lesson Module, Evaluation Engine, Database, Reward System.

Flow Summary:

Authenticate → fetch lesson → submit answers → evaluate → branch on correctness: provide feedback or award rewards → update progress → return to dashboard.

Data Updates:

Update Progress (lessonId, status, attempts, score); Update Rewards (points, badges); log evaluation results.

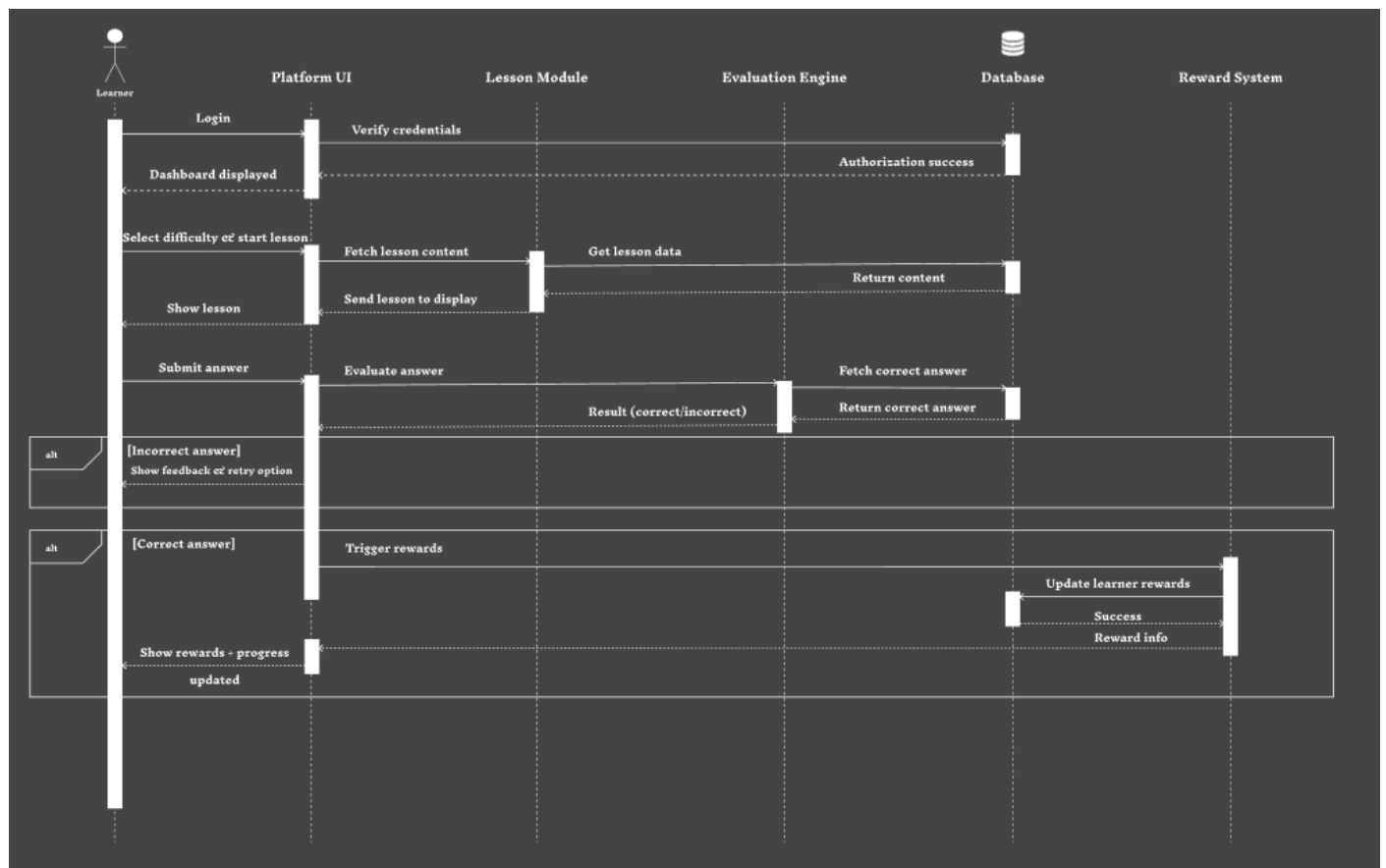


Figure 3.7.2.1: Sequence Diagram of Lesson completion, evaluation and rewards

4. Test Plan

4.1 Test Plan Overview

This test plan defines the strategy used to verify that the gamified micro-learning platform meets its functional and non-functional requirements. The objective is to ensure correctness, reliability, security, and usability across all core system features.

Testing focuses on backend APIs, frontend interactions, role-based access control, and critical learning workflows, including payment-based unlocking and lesson completion.

4.2 Testing Scope

In Scope

- User authentication and authorization
- Lesson, quiz, and quest workflows
- Payment-based unlocking of restricted content
- Progress tracking and reward allocation
- Admin content management features
- API validation and error handling

Out of Scope

- Third-party payment provider internal logic
- Load testing at production scale
- Platform-specific UI rendering differences

4.3 Testing Approach

Test Type	Description
Unit Testing	Validation of backend controllers, services, and models
Integration Testing	Verification of interactions between frontend, backend, and database
Functional Testing	End-to-end validation of user workflows
Security Testing	Authentication, authorization, and access control checks
User Acceptance Testing (UAT)	Manual testing based on user scenarios

Tools used:

- **Postman** – API testing
- **MongoDB Compass** – Database verification

4.4 Test Environment

Component	Specification
Frontend	React (local development server)
Backend	Node.js + Express
Database	MongoDB (local/Atlas)
Payment	Sandbox environment

Component	Specification
OS	Windows 10

4.5 Test Cases (Core Functional Tests)

These are a few test cases that were obvious since the beginning to test:

TC-01: User Login

Field	Description
Objective	Verify registered users can log in
Precondition	User account exists
Input	Valid email and password
Expected Result	User is authenticated and JWT token is issued

TC-02: Access Beginner Lesson (Free)

Field	Description
Objective	Verify free lessons are accessible
Precondition	User logged in
Input	Select beginner lesson
Expected Result	Lesson loads without payment

TC-03: Access Restricted Lesson Without Payment

Field	Description
Objective	Prevent unauthorized access
Precondition	User logged in
Input	Select intermediate/advanced lesson
Expected Result	Access denied and payment prompt shown

TC-04: Payment-Based Unlocking

Field	Description
Objective	Verify paid content unlocking
Precondition	User logged in
Input	Complete payment flow
Expected Result	Lesson unlocked and payment recorded

TC-05: Lesson Completion (Correct Answer)

Field	Description
Objective	Validate lesson success flow
Precondition	Lesson unlocked
Input	Correct quiz answers

Field	Description
Expected Result	Progress updated and rewards awarded

TC-06: Lesson Completion (Incorrect Answer)

Field	Description
Objective	Validate retry logic
Precondition	Lesson unlocked
Input	Incorrect quiz answers
Expected Result	Feedback shown and retry option provided

TC-07: Admin Creates Lesson

Field	Description
Objective	Verify admin content creation
Precondition	Admin authenticated
Input	Lesson data submission
Expected Result	Lesson stored in database

TC-08: Role-Based Access Control

Field	Description
Objective	Prevent privilege escalation
Precondition	Learner logged in
Input	Access admin endpoint
Expected Result	Access denied (403)

These are the basic tests that have been planned but more tests cases will be added in the future.

4.6 Test Execution Status

At the time of submission, some core test cases have already been executed during development and debugging phases. These tests validate critical workflows such as authentication, lesson access, and payment-based unlocking.

TC-01: User Login

Objective: Verify registered users can log in

Precondition: user has been registered

Inputs: user email and password

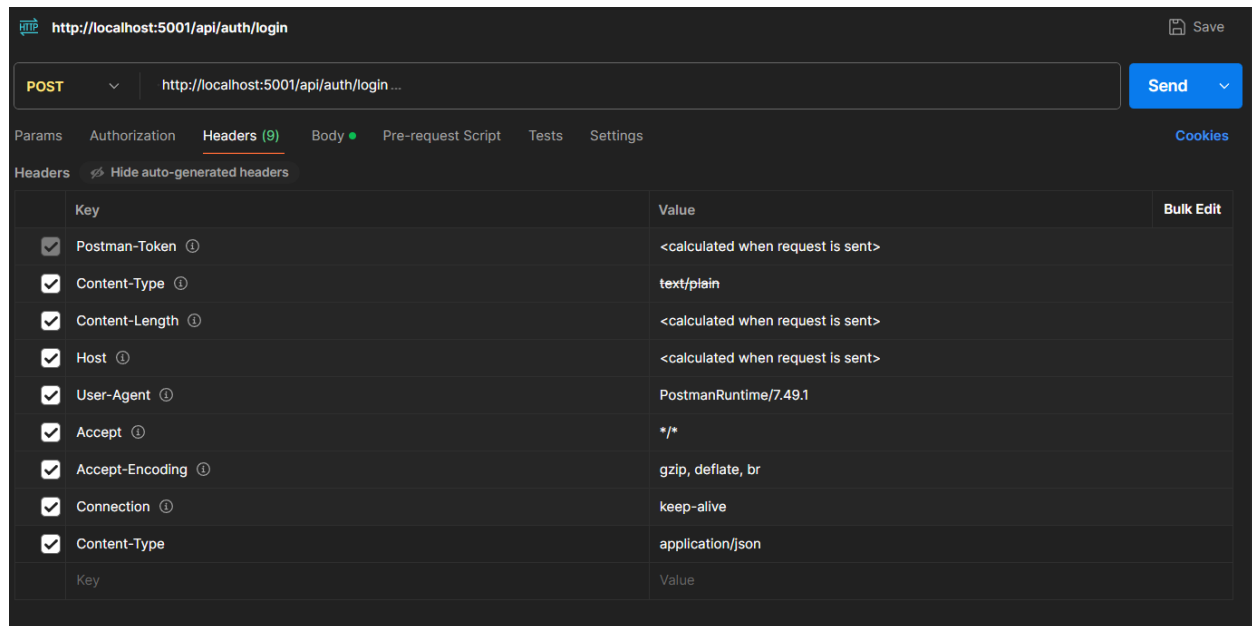


Figure 4.6.1: Headers used to test the API endpoint

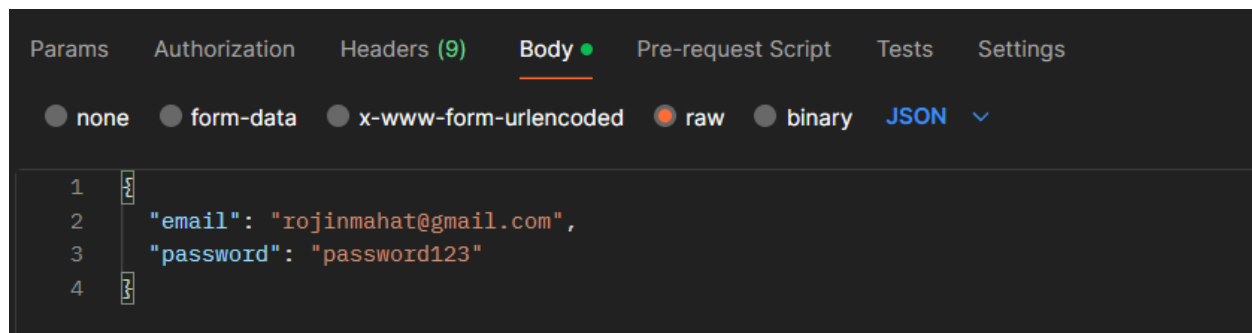


Figure 4.6.2: Body text with user credentials

Figure 4.6.4: existence of logged used in the database

TC-07: Admin Creates Lesson

Objective: Verify admins can add quests

Precondition: Admin is authenticated

Input: lesson content

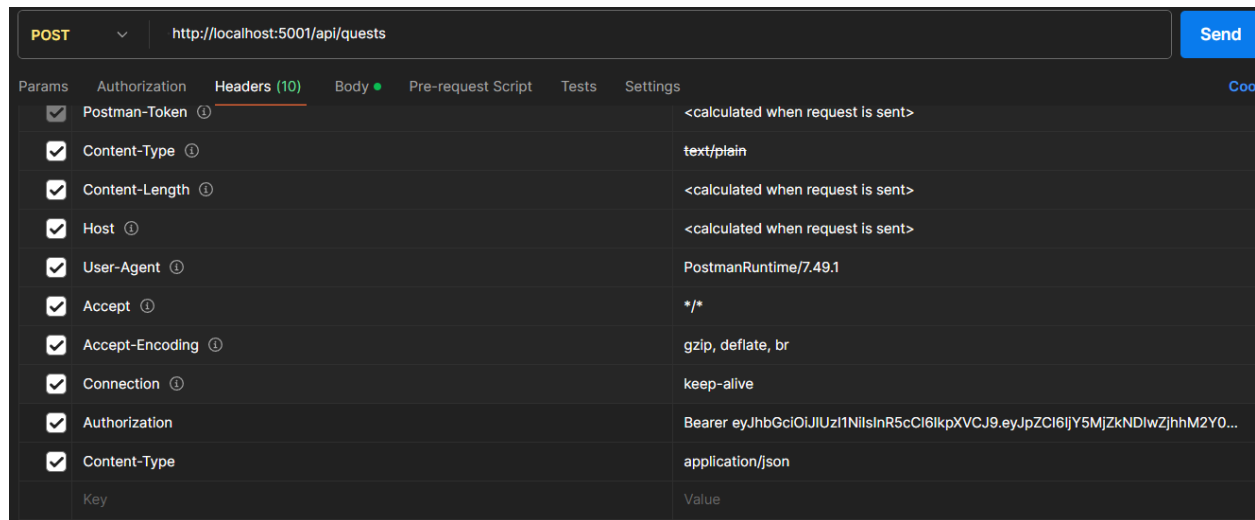


Figure 4.6.5: sending admin's bearer token to only allow admin access

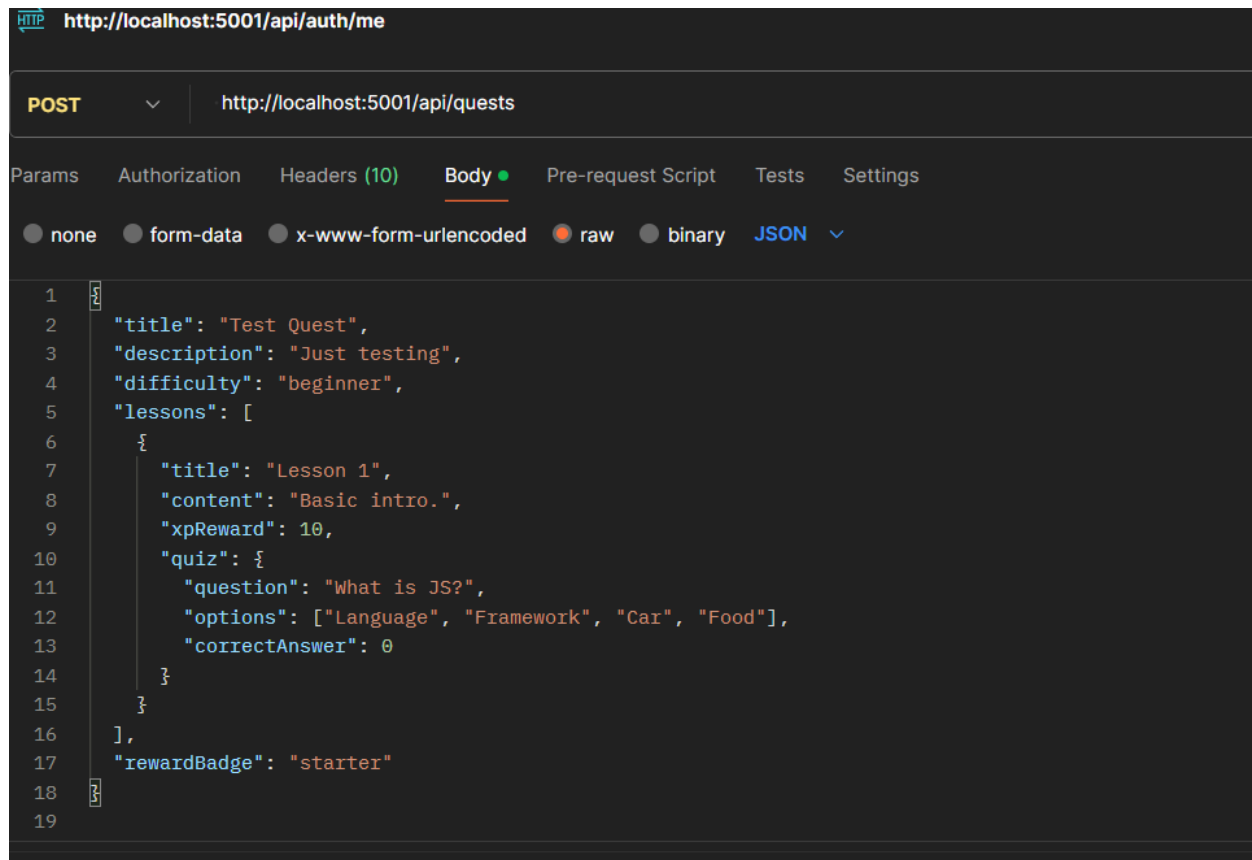


Figure 4.6.6: sending body of lesson content inside a quest

Result:

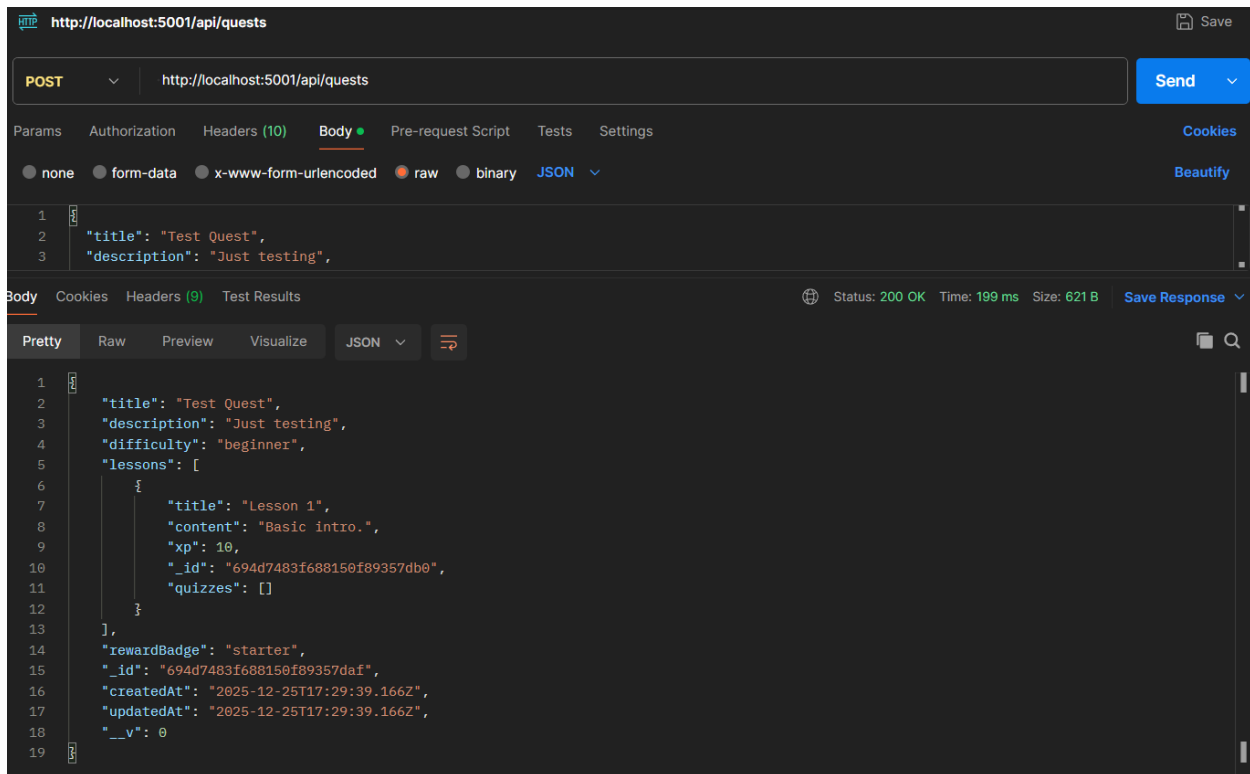


Figure 4.7.7: successfully added the content

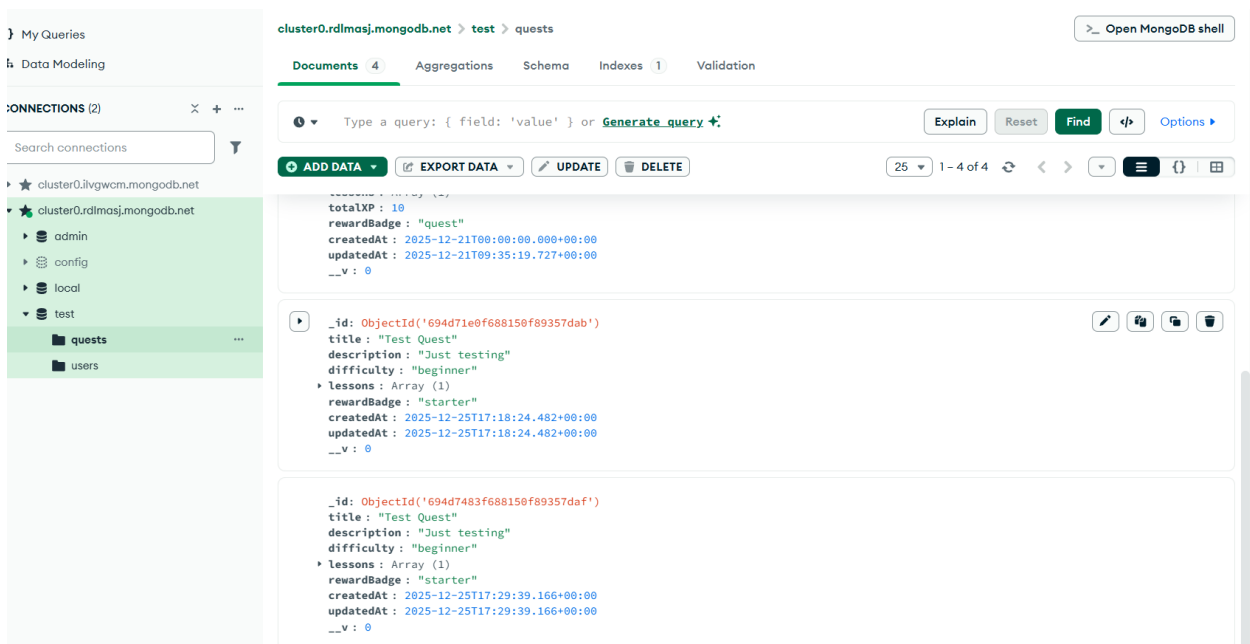


Figure 4.7.8: existence of added quest in the database

TC-08: Role-Based Access Control

Objective: Verify learners cannot access admin endpoints

Precondition: Learner is logged in

Input: Access admin endpoint

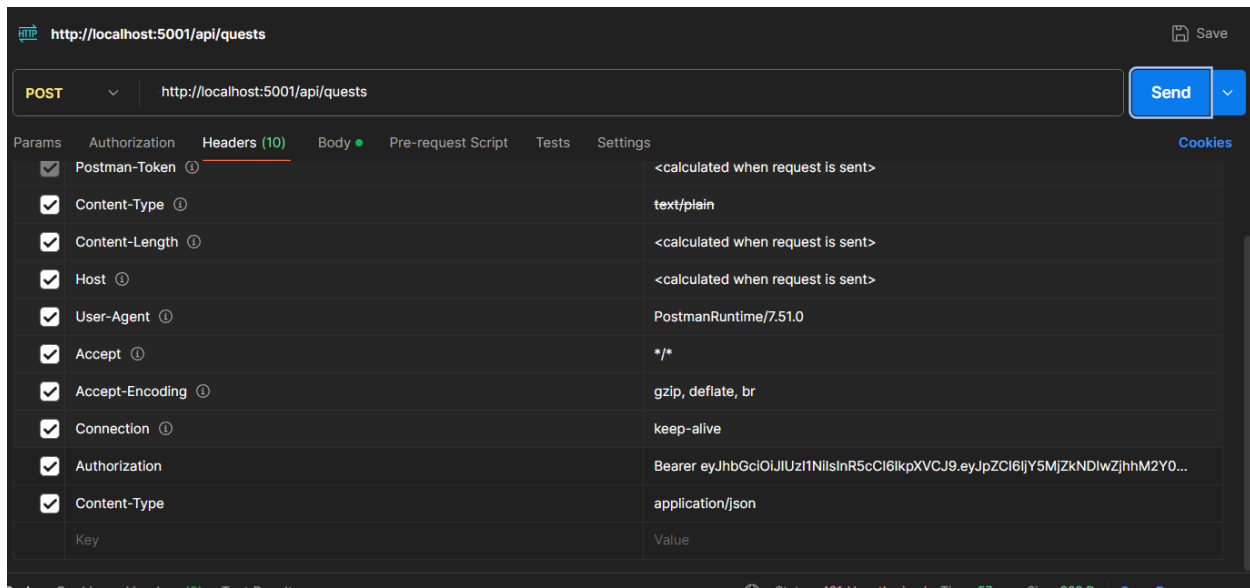


Figure 4.8.9: headers

The screenshot above displays headers. The Authorization header sends a non-admin bearer token to the back-end.

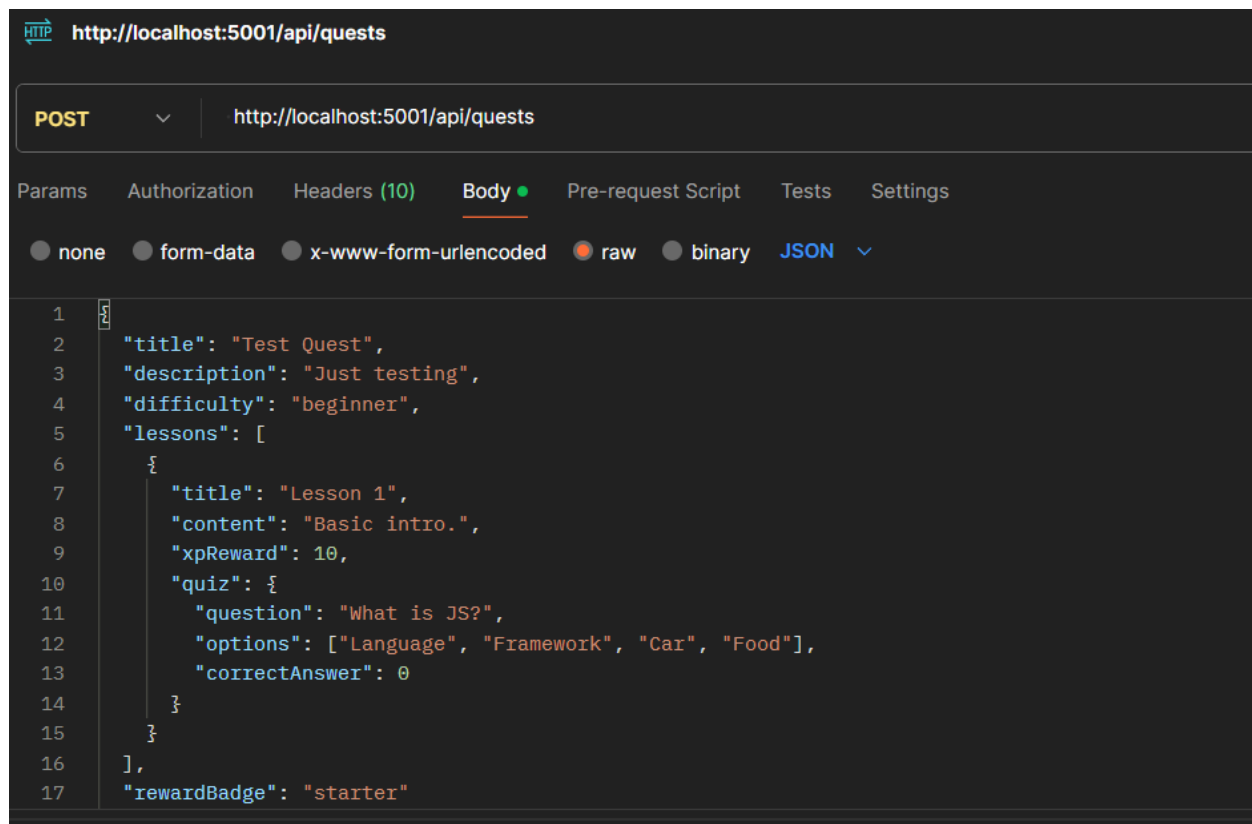


Figure 4.8.10: body containing basic lesson format

The screenshot displays basic format that the backend accepts for each quest.

Result:

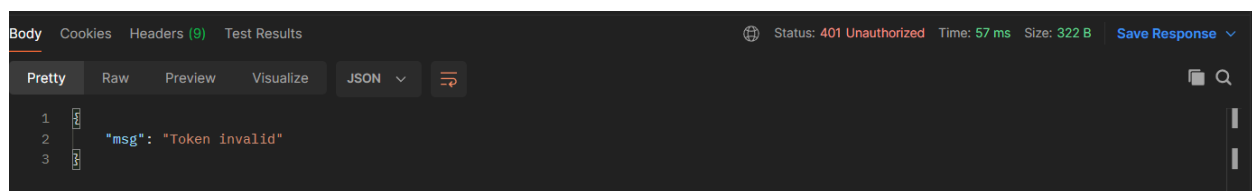


Figure 4.8.11: response from backend

The backend sends a 401 Unauthorized error with a message saying the token was invalid. Test successful.

5. Work Breakdown Structure

Phase	Tasks	Status
Planning & Requirements	Project scope and objectives	Completed
	Functional and non-functional requirements	Completed
	Artefact design and system diagrams	In Progress
Design & Architecture	Overall system architecture (MERN stack)	Completed
	Database schema and ERD	Completed
	API endpoint specification and design	Completed
	Security and authentication design	Pending
Backend Development	User authentication module (JWT, Google OAuth)	Completed
	Core data models (User, Quest, Lesson, Progress)	Completed
	Quest and Lesson CRUD API routes	Completed
	Payment integration and webhook handling	In Progress
	Gamification engine (XP, badges, levels)	Pending
	Email verification service (SMTP)	Pending
Frontend Development	Landing page and authentication views	Completed
	User dashboard and statistics view	Completed

Phase	Tasks	Status
	Admin panel for quest and lesson management	Completed
	Quest browsing and search functionality	Completed
	Lesson player and quiz interface	In Progress
	Payment checkout and content unlock flow	In Progress
Content & Gamification	Microskill quest design (Git, JavaScript, SQL, Docker)	Pending
	Lesson and quiz content authoring	Pending
	Badge visuals and reward structure	Pending
Testing & Quality Assurance	Backend unit and integration testing	Pending
	API endpoint testing	Pending
	UI and acceptance testing	Pending
	Payment flow and security testing	Pending
Deployment & DevOps	Backend build optimisation	Pending
	Frontend build and deployment (Vercel)	Pending
	MongoDB Atlas setup and data migration	Pending
	Environment configuration and CI/CD pipeline	Pending

Phase	Tasks	Status
	Domain, SSL and webhook configuration	Pending
	Monitoring and logging setup	Pending
Documentation & Handover	Artefact design report (final draft)	In Progress
	API documentation and usage examples	Pending
	Operations runbook and setup guide	Pending
	Final project report and presentation	Pending

The project is currently in an advanced implementation phase, with core architecture, backend services, and major frontend components completed. Remaining work primarily focuses on content creation, gamification logic, testing, and deployment, which will be addressed iteratively in the final development phase.